

saz-git

#platform/git
#target/local
#cat/PRIVESC
#cat/PERSIST
#cat/AZIZI-Sajjad
#tag/git

saz-git

Git - Initialiser un nouveau repo

```
git init
```

Git - Configurer le nom utilisateur

```
git config user.name "<User_Name>"
```

Git - Configurer l'email

```
git config user.email "<adresse_Mail>"
```

—
PROF

Git - Cloner un repo via SSH

```
git clone <ssh_link_to_repo>
```

Git - Cloner un repo via HTTPS

```
git clone <https_link_to_repo>
```

Git - Vérifier le status du repo

```
git status
```

Git - Voir les fichiers trackés

```
git ls-files
```

Git - Ajouter tous les fichiers au staging

```
git add .
```

Git - Ajouter un fichier spécifique au staging

```
git add <file_Name_or_path_to_file>
```

Git - Ajouter un dossier au staging

```
git add <dossier>
```

Git - Retirer tous les fichiers du staging

```
git reset
```

Git - Retirer un fichier du staging

```
git reset <file_Name_or_path_to_file>
```

Git - Retirer le dernier commit du staging

```
git reset HEAD~1
```

Git - Commiter les changements

```
git commit -m "<Message_du_commit>"
```

Git - Modifier et commiter d'un coup

```
git commit -am "<Message_du_commit>"
```

Git - Voir l'historique des commits

```
git log
```

Git - Voir l'historique en une ligne

```
git log --oneline
```

Git - Voir l'historique avec un graphique

```
git log --graph --all --decorate
```

Git - Voir l'historique avec les diffs

```
git log --all -p
```

Git - Voir les changements non commités

```
git diff
```

Git - Voir les changements d'un fichier spécifique

```
git diff <file_Name_or_path_to_file>
```

Git - Voir les changements en staging

```
git diff --staged
```

Git - Voir la différence entre deux commits

```
git diff <commit1> <commit2>
```

Git - Voir la différence entre HEAD et le commit précédent

```
git diff HEAD~1 HEAD
```

Git - Afficher un commit spécifique

```
git show commit-hash
```

Git - Afficher le dernier commit

```
git show HEAD
```

Git - Créer une nouvelle branche

```
git branch <branch_Name>
```

Git - Créer et basculer vers une nouvelle branche

```
git switch -c <branch_Name>
```

Git - Créer et basculer via checkout

```
git checkout -b <branch_Name>
```

Git - Lister les branches locales

```
git branch
```

Git - Lister toutes les branches

```
git branch -a
```

Git - Lister les branches avec le tracking

```
git branch -vv
```

Git - Basculer vers une branche

```
git switch <branch_Name>
```

Git - Basculer via checkout

```
git checkout <branch_Name>
```

Git - Supprimer une branche

```
git branch -d <branch_Name>
```

Git - Forcer la suppression d'une branche

```
git branch -D <branch_Name>
```

Git - Vérifier la branche courante

```
git branch
```

Git - Voir le status avant de commiter

```
git status
```

Git - Commiter sur la branche courante

```
git commit -m "<Commit_Message>"
```

Git - Pousser les changements

```
git push
```

Git - Pousser une branche pour la première fois

```
git push -u origin <branch_Name>
```

Git - Créer une branche et commiter

```
git switch -c <branch_Name>
```

Git - Ajouter tous les changements

```
git add .
```

Git - Créer un commit

```
git commit -m "<Commit_Message>"
```

Git - Pousser la nouvelle branche

```
git push -u origin <branch_Name>
```

Git - Corriger l'erreur upstream

```
git push --set-upstream origin <branch_Name>
```

Git - Pousser les changements après le premier push

```
git push
```

Git - Fusionner une branche

```
git merge <branch_Name>
```

Git - Rebase une branche

```
git rebase main
```

Git - Basculer vers main avant le rebase

```
git switch main
```

Git - Fusionner après un rebase

```
git merge <branch_Name>
```

Git - Annuler le dernier commit (soft)

```
git reset --soft HEAD~1
```

Git - Annuler le dernier commit (mixed)

```
git reset --mixed HEAD~1
```

Git - Annuler le dernier commit (hard)

```
git reset --hard HEAD~1
```

Git - Modifier le message du dernier commit

```
git commit --amend -m "<Commit_Message>"
```

Git - Récupérer les changements du remote

```
git fetch
```

Git - Récupérer depuis un remote spécifique

```
git fetch origin
```

Git - Récupérer et fusionner

```
git pull
```

Git - Récupérer et fusionner depuis main

```
git pull origin main
```

Git - Pousser vers main

```
git push origin main
```

Git - Pousser une branche spécifique

```
git push origin <branch_Name>
```

Git - Pousser en force

```
git push --force
```


Git - Pousser en force avec sécurité

```
git push --force-with-lease origin main
```

Git - Mettre en stash les changements

```
git stash
```

Git - Mettre en stash avec une description

```
git stash save "<Description_du_stash>"
```

Git - Voir la liste des stash

```
git stash list
```

Git - Appliquer et supprimer un stash

```
git stash pop
```

Git - Appliquer un stash spécifique

```
git stash apply stash@{0}
```

Git - Supprimer un stash

```
git stash drop stash@{0}
```

Git - Chercher un commit avec un keyword

```
git log --grep="keyword"
```

Git - Chercher dans l'historique avec grep

```
git log --oneline | grep "keyword"
```

Git - Trouver qui a modifié une ligne

```
git blame <file_Name_or_path_to_file>
```

Git - Trouver qui a modifié une plage de lignes

```
git blame -L 10,20 <file_Name_or_path_to_file>
```

Git - Voir les branches fusionnées

```
git branch --merged
```

Git - Voir les branches non fusionnées

```
git branch --no-merged
```

Git - Renommer la branche courante

```
git branch -m <new_Name>
```

Git - Renommer une branche spécifique

```
git branch -m ancien-nom <new_Name>
```

Git - Nettoyer les branches supprimées du remote

```
git fetch --prune
```

Git - Nettoyer avec prune

```
git remote prune origin
```

Git - Ajouter un remote

```
git remote add origin <ssh_link_to_repo>
```

Git - Ajouter un remote upstream

```
git remote add upstream https://github.com/autre/repo.git
```

Git - Voir les remotes

```
git remote
```

Git - Voir les remotes avec les URLs

```
git remote -v
```

Git - Changer l'URL d'un remote

```
git remote set-url origin git@github.com:nouvel-utilisateur/repo.git
```

Git - Supprimer un remote

```
git remote remove origin
```

Git - Ajouter des règles au gitignore

```
cat >> .gitignore << 'EOF'  
*.log  
node_modules/
```

```
.env  
EOF
```

Git - Vérifier ce que Git ignore

```
git check-ignore -v <file_Name_or_path_to_file>
```

Git - Vérifier les fichiers ignorés dans un dossier

```
git check-ignore -v terraform/*
```

Git - Voir les fichiers non trackés

```
git status --porcelain
```

Git - Voir les fichiers non trackés en détail

```
git ls-files --others --exclude-standard
```

Git - Forcer le tracking d'un fichier ignoré

```
git add -f fichier.log
```

PROF

Git - Supprimer un fichier du repo sans le supprimer localement

```
git rm --cached <file_Name_or_path_to_file>
```

Git - Ajouter les fichiers supprimés

```
git add -u
```

Git - Voir les changements par auteur

```
git log --author="<User_Name>"
```

Git - Voir le résumé des commits par auteur

```
git shortlog -sn
```

Git - Configurer le nom globalement

```
git config --global user.name "<User_Name>"
```

Git - Configurer l'email globalement

```
git config --global user.email "<adresse_Mail>"
```

Git - Voir la configuration

```
git config --list
```

Git - Installation de git-filter-repo

```
pip3 install --user git-filter-repo
```

PROF

Git - Ajouter git-filter-repo au PATH

```
export PATH="$HOME/.local/bin:$PATH"
```

Git - Vérifier la version de git-filter-repo

```
git filter-repo --version
```

Git - Purger des fichiers spécifiques

```
git filter-repo --force \  
  --path peojets/basic/aws.tfvars \  
  --path peojets/adv-1/aws.tfvars \  
  --path peojets/adv-2/aws.tfvars \  
  --path peojets/adv-3/aws.tfvars \  
  --path peojets/adv-4/aws.tfvars \  
  --path peojets/adv-301/aws.tfvars \  
  --invert-paths
```

Git - Purger tous les fichiers .tfvars

```
git filter-repo --force --filename-pattern '*.tfvars' --invert-paths
```

Git - Chercher les secrets AKIA

```
git log --all -p --diff-filter=A -- '*.tfvars' | grep -B 5  
"AKIA\|access_key"
```

Git - Vérifier que l'historique est clean

```
git log --all -p | grep -i "AKIA\|access_key\|secret_key" || echo  
"Clean"
```

Git - Lister tous les fichiers .tfvars dans l'historique

```
git log --all --diff-filter=A --name-only --pretty=format: -- '*.tfvars'  
| sort -u
```

Git - Rajouter le remote après filter-repo

```
git remote add origin git@github.com:utilisateur/visiplus-terraform-  
projets-perso.git
```

Git - Pousser en force après filter-repo

```
git push --force origin main
```

Git - Copier aws.tfvars en aws.tfvars.example

```
for dir in projets/adv-1 projets/adv-2 projets/adv-3 projets/adv-4
projets/basic; do
  cp "$dir/aws.tfvars" "$dir/aws.tfvars.example"
  sed -i 's/=.*/="CHANGE_ME"/' "$dir/aws.tfvars.example"
done
```

Git - Copier terraform.tfvars en exemple

```
cp projets/adv-2/terraform.tfvars projets/adv-2/terraform.tfvars.example
sed -i 's/=.*/="CHANGE_ME"/' projets/adv-2/terraform.tfvars.example
```

Git - Ajouter les règles .gitignore pour Terraform

```
cat >> .gitignore << 'EOF'
*.tfvars
*.tfstate
*.tfstate.*
.terraform/
EOF
```

Git - commit sur une branche déjà existante

```
git branch
git switch <branch_Name>
git status
git add .
git commit -m "<Commit_Message>"
git push
```

Git - premier push d'une branche

```
git push -u origin <branch_Name>
```

Git - créer une branche et commit dessus

```
git switch -c <branch_Name>
git status
git add .
git commit -m "<Commit_Message>"
git push -u origin <branch_Name>
```

Git - vérifier la branche distante liée

```
git branch -vv
```

Git - corriger l'erreur upstream

```
git push --set-upstream origin <branch_Name>
```

Git - workflow après le premier push

```
git status
git add .
git commit -m "Autre modification"
git push
```